AI ENGINEER · DATA · WEEK 1, DAY 1

Foundations of the data plane

Ingestion, chunking and embeddings for enterprise AI pipelines

ITC Infotech · AI and Data Engineering cohort

0

Introduction

What this course is, how each session runs, and the question we answer for six weeks

How each session runs

1

Concept

Short. Enough to make the choice you are about to face make sense.

2

Demo

I show it failing before I show it working. The failure is the lesson.

3

Lab

You build, in pairs, in breakout rooms. This is most of every session.

4

Debrief

We compare what broke. Protected time: I will not let the lab eat it.

What I need from you

Cameras on if your bandwidth allows. Questions in chat any time, no need to wait. Speak up the moment something does not land; in a virtual room I cannot see confusion.

Housekeeping

Two ten-minute breaks per session. Slides and a written reference go out after each day. Labs are in pairs, and I will rotate the pairs across weeks.

HOW THE SIX WEEKS RUN

Twelve days, one system

One concept, one failure shown live, one lab in pairs, one debrief. The same customer question runs through all twelve days.

WEEK 1 · FIND IT	WEEK 2 · FIND THE RIGHT ONE	WEEK 3 · GET THE NUMBER	WEEK 4 · TRUST IT	WEEK 5 · RUN IT NIGHTLY	WEEK 6 · SHIP IT SAFELY
DAY 1 **Embeddings and chunking** Lab 1: load seven files in five formats and find the two silent failures	DAY 3 **Hybrid retrieval and reranking** Lab 3: measure what each stage adds, and what it costs in seconds	DAY 5 **Questions into SQL, safely** Lab 5: accuracy you can quote, and every failure named	DAY 7 **Lineage and data quality** Lab 7: trace a wrong number to the column it came from	DAY 9 **Incremental pipelines** Lab 9: re-do only what changed, survive a crash, drop deletions	DAY 11 **PII, entitlements, audit** Lab 11: same question, two staff, two different answers
DAY 2 **Vector stores, recall, latency** Lab 2: benchmark three stores, then defend one in a memo	DAY 4 **Grounding and citations** Lab 4: an answer service that cites its sources and refuses	DAY 6 **Semantic layer and narration** Lab 6: metrics defined once, charts that do not mislead	DAY 8 **Quality gates in delivery** Lab 8: block a change that quietly makes answers worse	DAY 10 **Evaluation, drift and cost** Lab 10: three faults, three different signals	DAY 12 **Capstone** Lab 12: one customer question end to end, with evidence

Every session: concept 25 min · a live failure · lab 75 min in pairs · debrief 15 min, protected. Slides and a written guide go out after each day.

Your ETL ended in a table. This ends in an answer.

BEFORE

ETL

structured and semi-structured data

Sources — Tables, CSV, JSON, events

Extract — Pull rows that already have fields

Transform — Clean, join, change types, aggregate

Load — Warehouse or lakehouse table

Query — SQL returns the exact rows

NOW

AI data pipeline

unstructured data

Documents — PDF, Word, slides, sheets, scans

Load — Turn each file into clean text

Chunk — Cut the text into passages

Embed — Turn each passage into numbers

Vector store — Index the numbers so nearest is fast

Retrieve + answer — Find passages, the model writes from them

Day 1 | Day 2 | Days 3 and 4

	ETL	AI DATA PIPELINE
What goes in	Rows and fields with a schema	Pages of text, tables and images, with no schema
Smallest unit	A row	A chunk: a passage whose size you choose
How it fails	Loudly: a bad join or type stops the job	**Quietly: the job succeeds and returns the wrong passage**
How you test it	Row counts, nulls, keys	Recall, and whether the answer cites the right passage

ASK THE ROOM Which of your ETL habits still work here, and which one breaks first? One line in chat.

The story we are going to follow

One question, six weeks

A customer messages support at a food delivery company:

"My biryani arrived spilled across the whole bag. I asked for a refund and the app says I am not eligible. Why?"

Priya on the support team has about thirty seconds to answer. To do it properly her assistant needs the current refund policy and what it says counts as spillage, the restaurant partner contract that decides who absorbs the cost, this customer's order and claim history, and certainty that Priya sees this customer and no one else.

On day one the system cannot answer it at all. By the end of week six it can, and you will have built every layer in between.

Before anything else, a game

I will type one word in chat. You type the first word it makes you think of. Do not think about it.



Round one

The word is BIRYANI.

Watch what happens: dozens of different words, and almost all of them land in the same neighbourhood.



Round two

The word is BANK.

This time the room splits cleanly in two, and neither half is wrong.



What just happened

You clustered words by meaning without being told any rule.

That clustering is exactly what a machine does today. It is the whole of Day 1.

And round two is the problem we spend six weeks solving: when meaning splits, the machine has to pick, and it does not know which half you meant.

It predicts the next word from what you show it

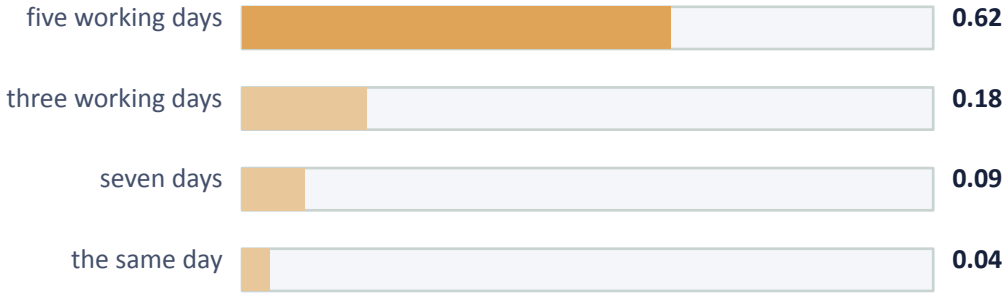
1. Your text is cut into tokens, pieces of words



2. The model reads all of them

Every token is compared with every other token, then the model scores what could come next

3. It scores the candidates and picks one



Then it adds that word to the text and does the whole thing again. That is the entire trick.

What it cannot do on its own

- **It has never seen your documents.**
  Your refund policy was not in its training data, and you cannot put 40,000 files in one question.
- **It does not know what changed yesterday.**
  Policies, prices and claims move daily. The model is frozen at training time.
- **It cannot show where an answer came from.**
  Without sources, nobody can check it, and nobody should sign it off.
- **It answers anyway.**
  Fluently and confidently, even when it is wrong. That is the expensive part.

Every lab you build fixes one of these four

Where we are going today

1

The failure

Why enterprise AI answers go wrong, and why the model is rarely the culprit.

2

Embeddings

How meaning becomes maths, and what that representation quietly loses.

3

Chunking

The single highest-leverage decision in your pipeline. Also the most neglected.

4

Loaders + Lab

Five file formats into one indexed corpus that you build yourself.

Two 10-minute breaks. Lab starts at the 2h30m mark, so the 15-minute debrief still fits.

- By the end of today you will have an indexed corpus you can query, built from five messy formats
- You will also have a list of everything that broke, which is the more valuable output

1

The failure

Why enterprise AI answers go wrong, and why the model is rarely the culprit

A STORY TO START

FreshCart shipped its AI support assistant. It was confidently, fluently wrong.

ONE SEARCH INDEX

refund_policy_2022.pdf

up to 2 claims

in 30 days

refund_policy_2024.pdf

up to 3 claims

in 30 days

Nothing said which one was in force.

Customer
Can I claim a refund again this month?

AI assistant
You have already used your two claims for this period, so this one is not eligible.

Quoted the 2022 limit

Refused

a refund she was entitled to

6 weeks

of blame on the model

0

things wrong with the model

ASK THE ROOM Whose fault was it? The model, the prompt, or something else? Answer in chat before we move on.

THE ONE IDEA FOR THIS WEEK

The model was never the bottleneck

If the right chunk never reaches the model, no prompt, no model upgrade, and no amount of temperature tuning will save the answer.



What teams debug

Prompt wording. Model choice. Temperature. System messages. Weeks of it.



What actually broke

Retrieval. The wrong document came back, or the right one never did.



What we build

A pipeline where you can prove which chunk was used, and why.

What happens between Priya's message and the answer

CUSTOMER, VIA PRIYA
"My biryani arrived spilled across the whole bag. The app says I am not eligible for a refund. Why?"

1 Who is asking

Priya, support agent. She may see policies and this customer's claims, not partner contracts.

2 Split it

Two needs: the rule, which is text, and this customer's history, which is numbers.

3a Find the rule

Search the policy chunks by meaning and keyword. Keep what Priya may see and what was in force on the claim date. Top hit: 2022 refund policy, clause 3.

3b Get the facts

A metric definition becomes SQL on the warehouse. Customer 57: approved claims on 4 and 12 March 2024; this one, 21 March, is the third in 30 days.

4 Build the prompt

The question, the top chunks each with its id, and the facts from SQL. Nothing else goes in.

If retrieval brought back the wrong chunk, no model can fix it from here.

5 Model writes

Drafts the answer and cites a chunk id for every claim it makes.

6 Check before sending

Every cited id must be a chunk it was given. Nothing solid enough? It says: I could not find it.

TO PRIYA
Declined: it was the third claim in 30 days, and the refund policy in force on 21 March 2024 allowed two. [2022 refund policy, clause 3] [claims register]

WHAT PRIYA NEVER SEES
The partner contract. A supervisor asking the same question gets one more line: the restaurant pays when packaging fails.

WRITTEN DOWN, EVERY TIME
Who asked, the chunk ids used, tokens, cost, latency, refused or not. When an answer is questioned, this is the evidence.

ASK THE ROOM Priya says the answer is wrong. Which box do you open first, and what do you look for?

2

Data Ingestion : Loading five formats

From files on disk to one indexed corpus

FIVE FORMATS, FIVE WAYS TO LOSE INFORMATION

What breaks in each, and the check that catches it

PDF
text layer

WHAT BREAKS

Reading order scrambles on two-column pages

YOUR CHECK

Read three random pages out loud

PDF
scanned

WHAT BREAKS

Extracts nothing at all. A silent empty string

YOUR CHECK

Assert non-empty text per page

DOCX
Word

WHAT BREAKS

Tables, headers and footnotes flatten or vanish

YOUR CHECK

Open a file you know has a table

PPTX
slides

WHAT BREAKS

Text boxes come out in creation order, not reading order

YOUR CHECK

Compare one slide against its text

XLSX
sheets

WHAT BREAKS

Merged cells, two header rows, formulas as values

YOUR CHECK

Check the row count against the sheet

The scanned PDF is the one that costs you weeks, because nothing anywhere reports a failure.

The ingestion toolbox: which package does which job

1 Connect

Fetch files from where they live: SharePoint, S3, ADLS, Confluence, the web

- LlamaHub readers
- LangChain community loaders
- Airbyte
- Databricks Auto Loader

2 Parse by format

Fast and free. Gives text, loses layout, and returns nothing from a scan

- pypdf
- PyMuPDF
- pdfplumber
- python-docx
- python-pptx
- openpyxl
- pandas
- BeautifulSoup
- trafilatura

3 Understand layout

Keeps tables, columns and reading order, and reads scans. Slower, or paid per page

- Unstructured
- Docling
- Marker
- LlamaParse
- Azure Document Intelligence
- AWS Textract
- Google Document AI
- OCR: Tesseract, PaddleOCR

4 Wrap as documents

LangChain loaders
LlamaIndex readers

They call the packages on the left for you. PyPDFLoader is pypdf underneath; the PowerPoint and Excel loaders need Unstructured.

→

OUT

```
Document(
  text,
  metadata
)
```

ready to chunk

IN LAB 1 Our loaders are layer 2: pypdf, python-docx, python-pptx, openpyxl. Step 2 adds layer 4, then two layer 3 tools: Docling, free, and ai_parse_document, paid at $0.70 to $1.75 per 1,000 pages for pages like these. For the scan, the Azure track uses Azure Document Intelligence at about $1.50 per 1,000 pages.

ASK THE ROOM 40,000 files, and 5 percent are scans or table-heavy contracts. Which files go through layer 3, and who pays for it?

The same seven files, five readers

		Our loaders src/freshcart.py	LangChain PyPDFLoader, Docx2txtLoader	LlamaIndex SimpleDirectoryReader	Docling free, reads the layout	ai_parse_document paid, on Databricks
PDF	refund_policy_v2_2022.pdf	reads it	reads it	reads it	reads it	reads it
PDF	refund_policy_v3_2024.pdf	reads it	reads it	reads it	reads it	reads it
PDF	partner_agreement_extract.pdf	reads it	reads it	reads it	reads it	reads it
PDF scan	support_escalation_sop_scanned.pdf	0 chars, no error	0 chars, no error	0 chars, no error	reads it, OCR	reads it, OCR
DOCX	support_sla_matrix.docx	reads it	reads it	reads it	reads it	reads it
PPTX	support_qbr_q1_2024.pptx	reads it	needs a package	reads it	reads it	reads it
XLSX	refund_claims_march_2024.xlsx	reads it	needs a package	reads it	reads it	no spreadsheets

THREE JOBS A READER CAN DO

Find a reader for every format

LlamaIndex and Docling: 7 of 7. ai_parse_document: 6, no Excel. LangChain: 5.

Keep the table structure

The frameworks split the Word table into loose cells. Docling and ai_parse_document keep it.

Say when extraction failed

Our loaders and both frameworks return 0 characters for the scan, silently. The layout tools read it.

ASK THE ROOM Forty minutes, a folder of contracts, a demo on Friday. Which reader, and what goes in the ticket about what it cannot do?

LangChain vs LlamaIndex: two different jobs

Same question, same files. What changes is how much each framework decides for you.

LangChain: you assemble each part

Built for chaining model calls, tools and agents (LangGraph for agents that run many steps), traced in LangSmith.

```
QUESTION = "Who pays the refund when items are missing at handover?"
docs    = PyPDFDirectoryLoader("data/raw").load()
pieces = RecursiveCharacterTextSplitter(
    chunk_size=1000, chunk_overlap=200).split_documents(docs)
store  = InMemoryVectorStore.from_documents(
    pieces, OpenAIEmbeddings())
hits    = store.similarity_search(QUESTION, k=4)
context = "\n\n".join(d.page_content for d in hits)
answer = ChatOpenAI(model=MODEL).invoke(
    f"Answer only from:\n{context}\n\nQuestion: {QUESTION}")
```

Nine lines, imports left out. Every choice is on the page, so every choice is yours to change.

LlamaIndex: the defaults do more for you

Built for connecting your documents to a model: read, cut into pieces, index, retrieve. LlamaParse is its paid parser.

```
QUESTION = "Who pays the refund when items are missing at handover?"
docs    = SimpleDirectoryReader("data/raw").load_data()
index   = VectorStoreIndex.from_documents(docs)
answer = index.as_query_engine().query(QUESTION)
```

Three lines. The defaults chose for you: pieces of 1,024 tokens, the 2 closest pieces, no reranking, OpenAI models unless you set others.

Common pattern: LlamaIndex finds the right pieces, and a LangChain or LangGraph agent decides what to do with them.

WHAT LAB 1 MEASURED LlamaIndex read 7 of 7 files and LangChain 5 of 7. LlamaIndex took about a third of the time; LangChain kept the author and creation date. Both read PDFs with pypdf, and neither said the scan came back empty.

Enterprise data loaders: a selection matrix

Pick the loader by what the downstream RAG architecture needs, not by habit.

Use LangChain when

Simple formats where you need clean text fast, and a large set of ready integrations.

Example: Help-centre and FAQ pages (HTML)

```
BSHTMLLoader("help/refunds.html").load()
```

Text out in one line, straight into the splitter. Nothing structural to lose.

Use LlamaIndex when

Complex, nested documents where structure and metadata must survive into retrieval.

Example: 60-page restaurant partner contract with a cost-sharing table

```
LlamaParse().load_data("partner_contract.pdf")
```

Keeps the table intact and clause headers as metadata, so the answer can cite "clause 7.2".

Source data	Preferred loader	Framework	Primary engineering advantage
Complex PDFs, contracts	`LlamaParse`	**LlamaIndex**	Pulls out embedded tables, layout and headers cleanly.
Transaction logs (JSON)	`JSONLoader`	**LangChain**	Uses jq-schema path queries to parse nested objects.
Database tables (SQL)	`DatabaseReader`	**LlamaIndex**	Maps table schemas straight into queryable text nodes.
Internal wikis, HTML	`BSHTMLLoader`	**LangChain**	Uses BeautifulSoup to strip navigation and other noise.

Mixing is normal: pick per source. LlamaIndex converts LangChain documents with Document.from_langchain_format. Simpler PDFs: PyMuPDFLoader (LangChain).

3

Embeddings

How meaning becomes arithmetic, and what gets lost on the way

An analogy

The librarian who never reads the books

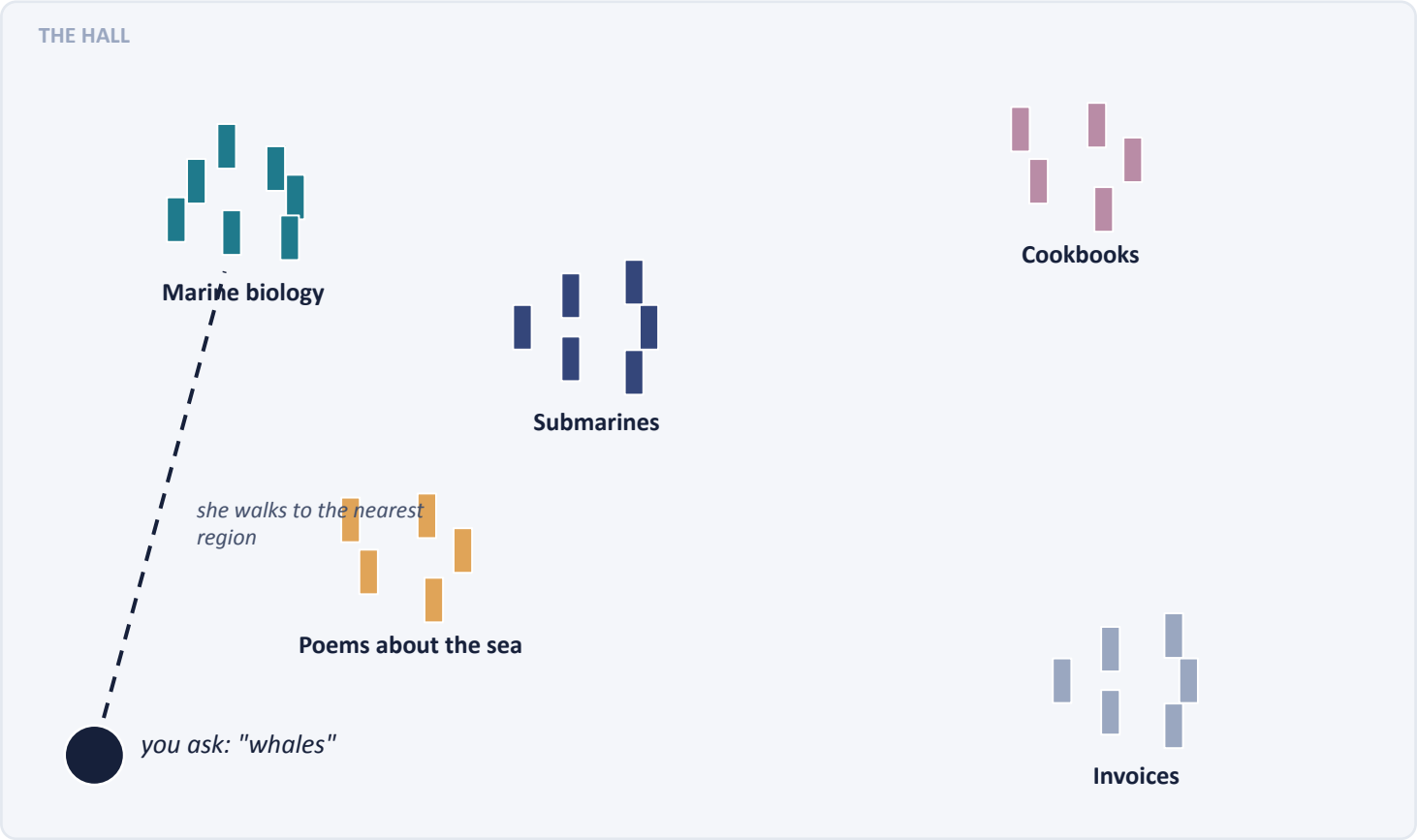
Imagine a librarian who cannot read. Instead, she places every book in a vast hall according to what it feels like it is about. Books on marine biology drift toward one corner. Books on submarines end up nearby, because they share words and themes. Poetry about the sea lands somewhere between.

Ask her for something on whales, and she walks to the right region of the hall and hands you what is closest. She has no idea whether the book is correct, current, or even relevant to your actual question. She only knows what sits near what.

That is an embedding model.

AN ANALOGY

The librarian who never reads the books



1 She shelves by feel

Every book goes where it seems to belong. Similar topics drift together.

2 She fetches the closest

Ask for whales and you get what sits nearest, instantly.

3 She never opens a book

Correct? Current? Relevant to your question? She cannot tell.

That is an embedding model: it knows nearness, not truth.

ASK THE ROOM Where in your company would "nearest" be dangerously different from "correct"?

Embedding, in one sentence

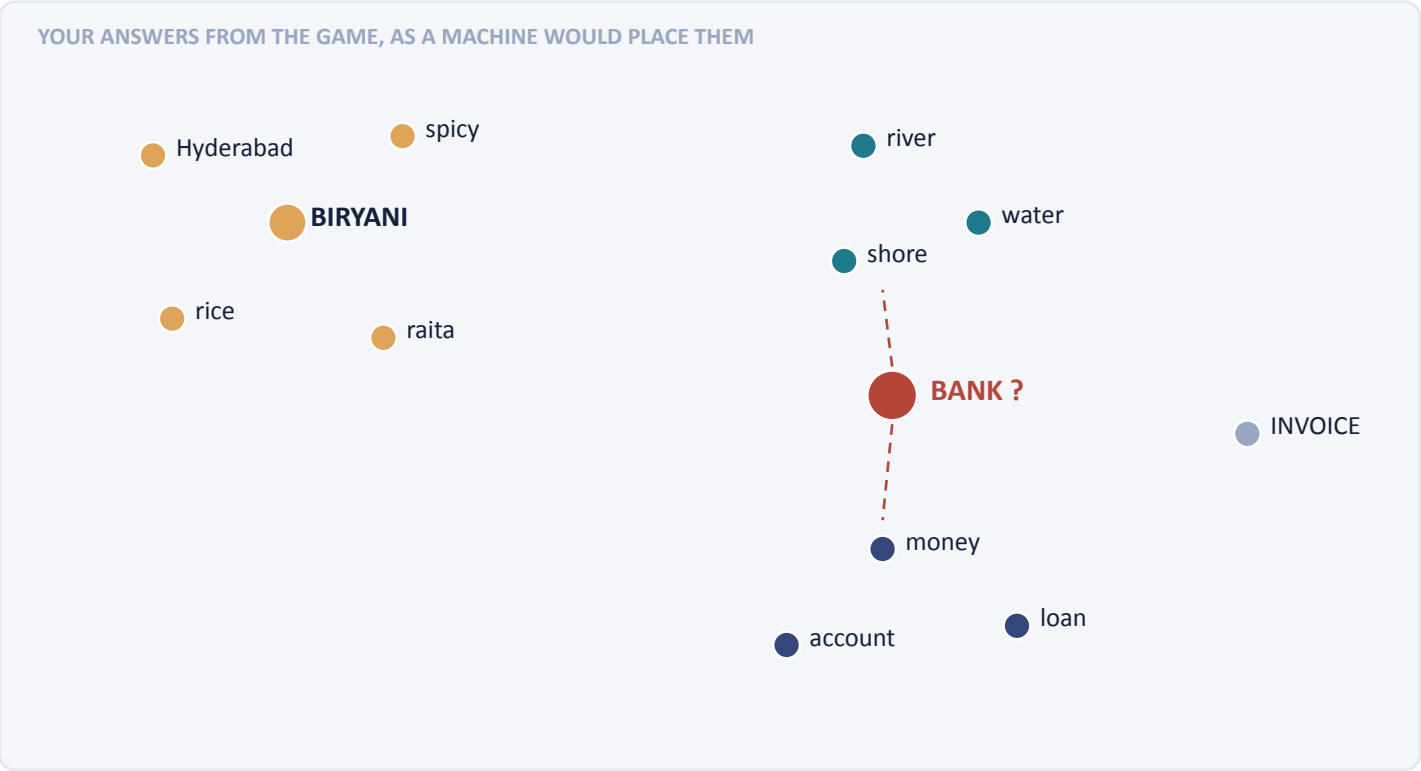
An embedding is text turned into a list of numbers, arranged so that things which mean similar things end up close together.

- You already did this in the word game. BIRYANI and RICE landed near each other; BIRYANI and INVOICE did not
- The machine does it with arithmetic instead of instinct, over millions of pieces of text
- Once text is numbers, "find me something similar" becomes "find the nearest numbers", which computers are very fast at

That is the entire trick. Everything after this is detail about how well it works and where it breaks.

Embedding: the word game, done with arithmetic

Text turned into a list of numbers, arranged so that things which mean similar things end up close together.



1 Meaning decides the spot

BIRYANI lands by rice and raita, not by words that look alike.

2 Distance means unrelated

INVOICE sits far away: nothing about it is near food.

3 One word, two meanings

BANK belongs near river AND near money. One dot cannot sit in two places.

ASK THE ROOM Where should BANK go: near river, near money, or in between? Vote in chat. The next two slides answer it.

EMBEDDING, IN ONE PICTURE

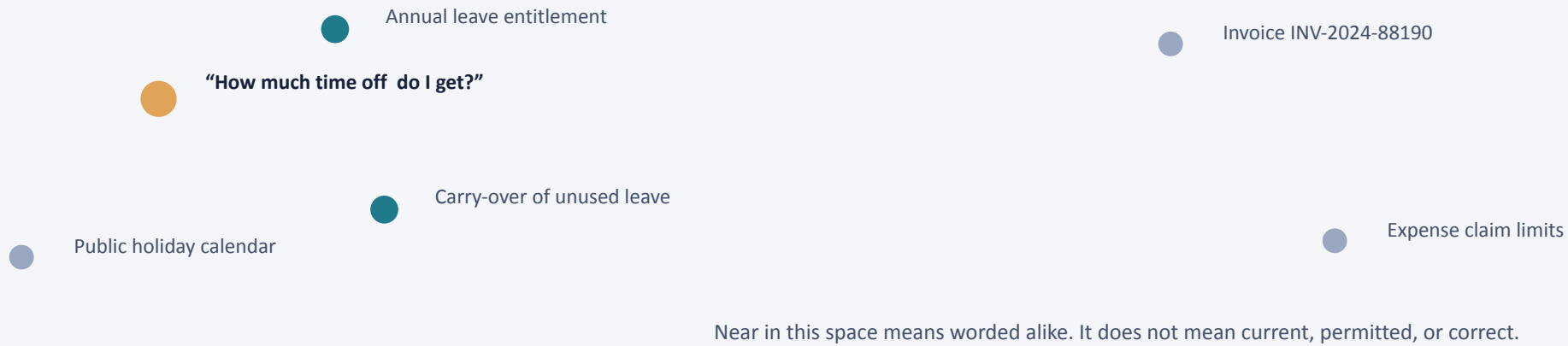
One sentence becomes one point in space

“How much time off do I get?”

embedding model
reads the words, outputs numbers

[0.021, -0.184,
0.339, ...]
1,536 numbers

meaning space, flattened to two dimensions



Near in this space means worded alike. It does not mean current, permitted, or correct.

Nothing of the original words is kept. The list of numbers is the whole representation.

THREE WORDS PEOPLE TREAT AS ONE

Similar, relevant and correct are three different questions

SIMILAR

Do these two pieces of text sit near each other?

Vector search measures this, and nothing else.

RELEVANT

Does this piece actually answer the question asked?

A reranker judges this. Week 2.

CORRECT

Is it true, current, and allowed for this reader?

No model checks this. Metadata, governance and people do.

FreshCart's two refund policies, as the machine saw them

the question: how many refund claims can a customer make in thirty days?

2022 policy: up to TWO claims in any rolling thirty day period	**similarity 0.94**	**withdrawn April 2024**
2024 policy: up to THREE claims in any rolling thirty day period	**similarity 0.80**	**in force**

The withdrawn policy scored higher. Similarity was working perfectly. Nobody had told it which one was live.

Where embeddings quietly fail in enterprises

Failure type	Example query	What goes wrong
Internal product codes	"MX-440 renewal terms"	Model never saw MX-440. Embeds it as near-noise
Acronyms with local meaning	"What is our TAT for KYC?"	TAT may embed toward the wrong domain entirely
Exact identifiers	"Invoice INV-2024-88190"	Semantic search is the wrong tool. You need keyword match
Domain jargon	"charge-off provisioning"	General models blur terms a specialist keeps distinct
Numbers and dates	"the 2023 revision"	Numerals carry almost no semantic signal

Every one of these is fixed the same way: do not rely on embeddings alone. Hybrid retrieval and metadata, both in Week 2.

- Ask yourself now: what are the three acronyms in your organisation that no general model could possibly know?
- Those are the queries you should test with first, not the easy ones

BEFORE WE TALK ABOUT LIMITS

What is a token, and how do you count one?

ONE SENTENCE, AS THE MODEL SEES IT

`My` `bir` `y` `ani` `arrived` `spilled` `across` `the` `whole` `bag` `.`

8 words = **11 tokens**

Common words are one token each. "biryani" is rare in the text the tokenizer learned from, so it is cut into 3 pieces. The model only ever sees the pieces.

RULE OF THUMB, ENGLISH

1 token ≈ 4 characters

≈ three quarters of a word

So 400 tokens ≈ 300 words ≈ one section of a policy

THE SAME WORD COUNT IS NOT THE SAME TOKEN COUNT

Plain English

"Customers may raise up to three refund claims in any rolling 30 day period."



16 tokens

14 words

An invoice code

`Invoice` `INV` `-` `202` `4` `-` `881` `90`



8 tokens

2 words

The same rule in Hindi

ग्राहक किसी भी 30 दिन की अवधि में तीन रिफंड दावे कर सकते हैं।



59 tokens

14 words

COUNT IT EXACTLY

```
import tiktoken
enc = tiktoken.get_encoding("cl100k_base")
len(enc.encode(text))    # 16
```

Use the tokenizer of the model you embed with. Other models count differently.

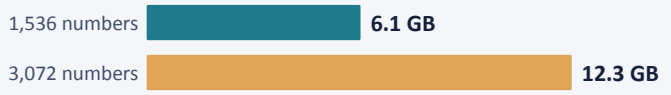
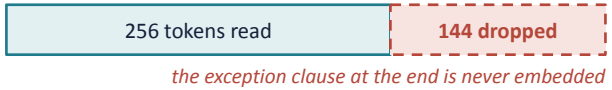
WHY YOU NEED THE COUNT

Model limit: past it, text is dropped (next slide)
Cost: you pay per token, not per word
Chunk size: set in tokens, in the chunking section

ASK THE ROOM Your documents are full of codes and Hindi names. Does 400 tokens hold more or fewer than 300 words?

FOUR AXES THAT ACTUALLY MATTER

Choosing an embedding model

THE AXIS	THE FRESHCART EXAMPLE	WHAT TO DO
1 Dimensions How many numbers per chunk. More numbers hold more detail and cost more to store.	1,000,000 chunks, 4 bytes per number: 	**Start at 1,536. Go bigger only if your own test says so.**
2 Context window The most text the model reads at once. Anything past it is dropped, with no error.	A 400-token refund chunk into all-MiniLM-L6-v2, which reads 256:  *the exception clause at the end is never embedded*	**text-embedding-3-small reads 8,191 tokens. Keep chunks well under the limit.**
3 Domain fit How well the model knows your words: codes, acronyms, jargon.	Take 50 real questions Priya answered last month. Mark the right passage for each. Count how often each model puts it in the top 5.  *right passage at #2: a hit*	**Your 50 questions beat any public leaderboard.**
4 Cost and locality The price of one full pass, and where the text is allowed to go.	One full pass, 1M chunks:  *Support chats hold phone numbers and addresses, so data rules decide where this runs.*	**Pick a hosted region the data rules allow, or run an open model yourself.**

FRESHCART'S PICK text-embedding-3-small: 1,536 numbers, reads 8,191 tokens, $8 a full pass. Switching later means re-embedding all 1,000,000 chunks, so test before you choose.

ASK THE ROOM Which of these four would decide the model at your company before quality even comes up?

Five kinds of embedding models

1 Dense

matches meaning



The whole passage becomes one list of numbers.

Ask **“my food came spilled”**: it finds the spillage claims section, though no word is shared.

e.g. text-embedding-3-small

2 Sparse

matches exact words



One number per word; almost all of them are zero.

Ask **“INV-2024-88190”**: it finds that one invoice. Dense returns any invoice that looks alike.

e.g. BM25, SPLADE

3 Multi-vector

checks every word



One list of numbers for each word.

Ask **“spilled biryani refund”**: the passage that matches all three words beats a general refund page.

e.g. ColBERT

4 Multimodal

matches pictures too



Text and images become the same kind of numbers.

Show **a photo of a soaked delivery bag**: it finds past spillage claims with similar photos.

e.g. CLIP, ColPali

5 Domain

knows one field’s words



A dense model, trained further on one field.

Ask **“chargeback limits”**: it finds chargeback rules, not charge-off rules.

e.g. voyage-finance-2

IN OUR LABS Dense only: bge-small in Lab 1, text-embedding-3-small on the Azure track. Week 2 adds sparse; using the two together is hybrid search.

ASK THE ROOM A customer writes "INV-2024-88190 arrived spilled". Which two kinds does it take to find the right answer?

OPERATIONAL REALITY

Embedding 40,000 files without paying twice

40,000
files in the corpus

→

~1,000,000
chunks, at about 25 per file

→

1,000
chunks per batch call

→

1 model
the same one for every chunk

→

Delta table
hash, vector, metadata

WHAT ONE FULL PASS COSTS

1,000,000 chunks × 400 tokens = 400 million tokens

`text-embedding-3-small`
$0.02 per 1M tokens



$8

`text-embedding-3-large`
$0.13 per 1M tokens



$52

List prices at time of writing. Check your own contract. The bill that grows is the one you pay twice.

1. **Fingerprint every chunk**
   Store a hash of its text. Unchanged hash, no new call. A nightly re-run then costs cents, not dollars.

2. **Batch, then back off**
   Send hundreds of chunks per request. When the service says slow down (HTTP 429), wait and retry.

3. **Run it as a Spark job**
   On Databricks: one batch per partition, results written to a Delta table keyed by the hash.

ASK THE ROOM Your nightly job re-embeds everything. One file changed. How much did you just pay, and how would you know?

QUICK CHECK

Before the break, three questions

1. A user searches "how many days off do I get" and the top result is titled "Annual leave entitlement", sharing no words with the query. What made that match possible?

2. Your corpus contains the string "Invoice INV-2024-88190". Why is semantic search a poor tool for finding it, and what would you reach for instead?

3. Six months from now a noticeably better embedding model is released. What does adopting it actually cost you?

ANSWERS

Before the break, three questions

1

The match with no shared words

Embeddings position text by meaning rather than by vocabulary, so a paraphrase lands near the original in vector space. High similarity with zero keyword overlap is exactly what the technique is for.

2

The invoice number

An identifier carries almost no semantic signal: the model never learned what INV-2024-88190 refers to, so it embeds close to noise. This needs exact keyword matching, which is why hybrid retrieval exists. Week 2.

3

The cost of a better model

Re-embedding the entire corpus. Vectors from two models are not comparable, so there is no partial migration. Add index rebuild time and a plan for serving traffic while it runs.

Break

10 minutes. When we come back: the decision that quietly determines whether any of this works.

4

Chunking

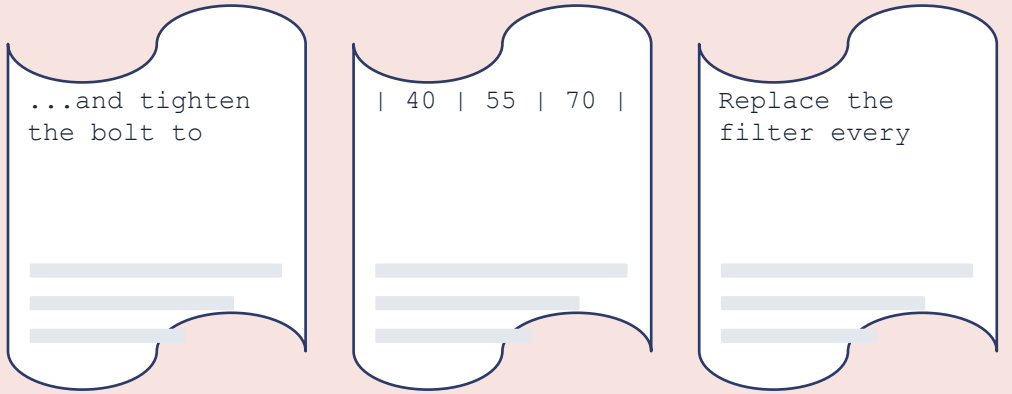
The highest-leverage decision nobody documents

AN ANALOGY

Tearing pages out of the manual

A 400-page equipment manual. The rule: hand the technician one page at a time.

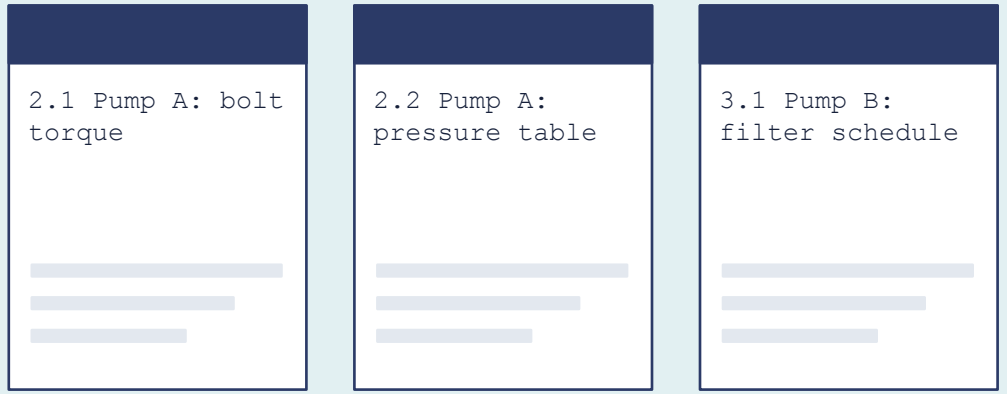
Tear every 500 words



- starts mid-sentence
- ends mid-table
- which machine?

Technically a page. Practically useless.

Tear on section boundaries



- has a heading
- finishes its thought
- names its machine

Each page answers a question on its own.

ASK THE ROOM Hands up: who has shipped a default chunk size without ever changing it? Same document, same model, different system.

Chunk, in one sentence

A chunk is one piece of a document, small enough to be useful on its own, and it is the unit your search actually returns.

- You never retrieve a whole document. You retrieve chunks, and the model sees only those
- So if the answer is split across two chunks, or a chunk lacks the heading that gave it meaning, the answer is gone
- Deciding where documents get cut is a design decision, not a default setting

This is the decision that most affects quality, and the one most teams never revisit.

A chunk is the only thing search ever returns

refund_policy_2024.pdf

1. Scope

2. Spillage and damage

2.1 Evidence within 24 h

3. Claim frequency limits

3.1 Exceptions

4. Refund method

5. Appeals

search picks the nearest chunk →

WHAT THE MODEL SEES

"3. A customer may make up to three refund claims in 30 days."

Nothing else. Not the exception in 3.1.

Split answer

A rule in one chunk, its exception in the next: only one comes back.

Lost heading

A chunk without its heading loses what it was about.

Your decision

Where documents get cut is design, not a default setting.

ASK THE ROOM A policy answer needs clause 3 AND its exception 3.1, in two different chunks. What does the customer get told?

Where the cut lands decides what you can answer

refund_policy_v3_2024.pdf, page 1

2.1 Spillage and damage

cut at 500

3. Claim frequency limits

4. Refund method and timing

chunk 2

"... photographic evidence must be submitted within 24 hours of delivery. 3. Claim frequency limits A customer may ma"

chunk 3

"ke up to three refund claims in any rolling thirty day period. A fourth and any subsequent claim within that period is automatically declined and must be escalated to …"

What just broke

- Chunk 3 starts in the middle of the word make.
- Neither piece names the policy version or its date.
- The heading sits in chunk 2, and the rule it introduces sits in chunk 3.
- Ask how many claims are allowed and you retrieve one, never both.
- Retrieved on its own, neither chunk can answer the question.

And nothing reports an error

Both chunks index cleanly, both get returned, and the model answers fluently from half a sentence. The only symptom is a wrong answer, days later, from a user who does not report it.

Five chunking strategies

1



Fixed size

Split every N characters or tokens.

Fast and predictable, but blind to structure: the cut lands mid-sentence, even mid-word.

2



Fixed size with overlap

The same cut, but each chunk repeats the last 10 to 15 percent of the one before.

A sentence cut at the edge arrives whole in one of the two. It costs 10 to 15 percent more storage.

3



Recursive

Try paragraph breaks first, then sentences, then characters.

The sensible default. Respects structure when it exists. Use it with overlap too.

4



Document structure

Split on the document's own markers: headings, sections, slides, rows.

Best quality when the structure is reliable.

Lab 1 calls it structural.

5



Semantic

Split where the topic shifts, detected by embedding similarity.

Expensive; sometimes worth it for long, unstructured prose.

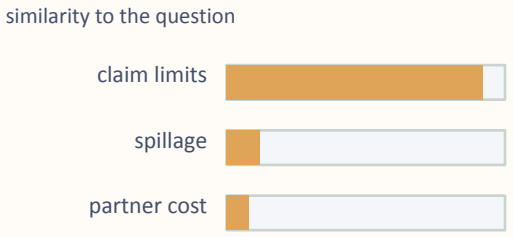
Start at 3, with 10 to 15 percent overlap. Move to 4 for any document type where the structure is dependable. Reach for 5 only when you can prove 3 and 4 are failing.

CHUNK SIZE

Precise and thin, or rich and vague

Small chunk, 150 tokens

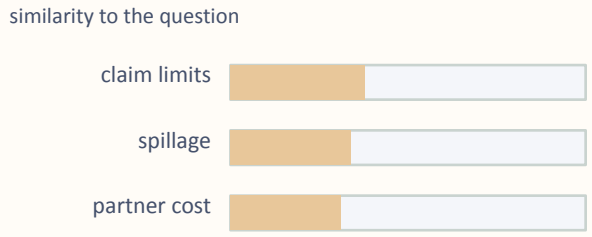
"A customer may make up to three refund claims in any rolling thirty day period."



One topic, so the vector points firmly at it. The risk: the heading, the date and the exception live in the chunks on either side.

Large chunk, 3,000 tokens

Claim limits, spillage rules, evidence deadlines, partner cost sharing, wallet credit, appeals ... all in one piece.



Six topics share one vector, so it is weakly similar to all of them and strongly similar to none. The answer is in there; retrieval just stops finding it first.

Where to start, and how to stop guessing

300 to 800 tokens for prose, 10 to 15 percent overlap, then measure on your own questions. There is no universally correct number, and anyone who gives you one has not seen your documents.

OVERLAP, IN ONE PICTURE

A sentence cut in half is lost to both pieces

WITHOUT OVERLAP

… photographic evidence must be submitted within 24 hours of delivery. 3. Claim frequency limits A customer may ma

ke up to three refund claims in any rolling thirty day period …

Neither piece can answer how many claims are allowed. Nothing reports an error.

WITH 10 TO 15 PERCENT OVERLAP

… photographic evidence must be submitted within 24 hours. 3. Claim frequency limits
A customer may make up to three refund claims in any rolling thirty day period.

A customer may make up to three refund claims in any rolling thirty day period. A fourth and any subsequent claim is automatically declined …

The boundary sentence appears in both, so whichever piece is retrieved carries the rule.

Too little overlap and you get the cut above. Too much and you pay for it twice: a bigger index, slower queries, and the same sentence retrieved as two near-identical results.

Typical setting: 10 to 15 percent of chunk size. It is the cheapest insurance in the pipeline.

Metadata: the cheapest quality win you will ever get

Provenance

Source file, page or slide number, author, system of record.

Without this you cannot cite, and a RAG answer without a citation is a rumour.

Recency and version

Effective date, version, superseded flag.

This single field would have prevented the 2022 policy answer.

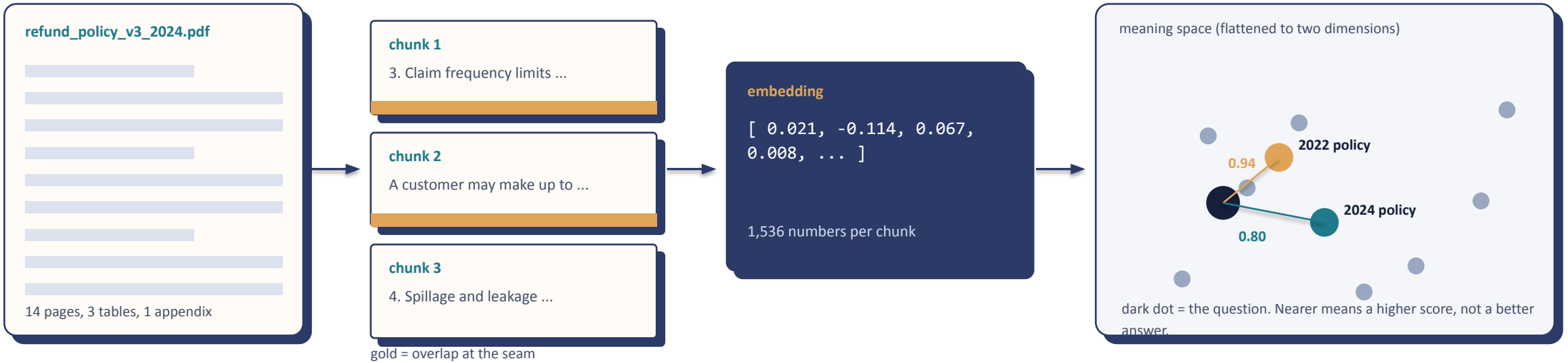
Access and scope

Department, sensitivity, entitlement group.

Permissions must be enforced at retrieval, never by asking the model to be discreet.

Attach metadata during ingestion. Retrofitting it later means re-processing the corpus, and by then it is a project rather than a line of code.

From a policy PDF to a point in space



Similar is not the same as correct

In the Day 1 lab the withdrawn 2022 policy scores 0.942 and the current one 0.795, for the same question. The fix is one metadata field (effective date) and a filter, not a bigger model.

Day 1 lab: seven files, five formats, two silent failures to find. One of them extracts zero characters and reports success.

QUICK CHECK

Three questions on chunking

1. A policy clause and its exception end up in two different chunks. A user asks something that needs both. What happens, and why does nothing report an error?

2. You raise chunk size from 400 to 3,000 tokens to give the model more context, and retrieval quality drops. Why?

3. Two teams index the same 10,000 documents with the same embedding model, and one gets noticeably better answers. Name two chunking decisions that could explain the gap.

Take answers in chat. Do not move on until at least two people have committed to an answer.

ANSWERS

Three questions on chunking

1

The split clause

Retrieval returns one chunk and the model answers fluently from half the picture. Nothing errors because a chunk that was never retrieved is invisible to the pipeline. The only symptom is a wrong answer.

2

Bigger chunks, worse results

Dilution. One vector now represents many topics at once, so it is weakly similar to all of them and strongly similar to none. Context went up and precision went down.

3

Same documents, different quality

Any two of: splitting on structure rather than fixed size, carrying headings and clause numbers into the chunk text, attaching source and date metadata, sensible overlap at boundaries, and choosing a strategy per document type instead of one rule for everything.

QUESTIONS THAT ALWAYS COME UP

Day 1: the ones worth answering before the lab

Why not put everything in a long context window?

Because you would pay for 40,000 documents on every question, wait for them, and still have no citation and no way to hide what a user may not see. Retrieval turns 40,000 files into five passages you can point at.

Why not fine-tune the model on our documents?

Fine-tuning teaches format and tone, not facts. Your policy changed last Tuesday; the weights did not. And a fine-tuned model still cannot show where an answer came from.

Which embedding model do we start with?

A general model of 768 to 1,536 dimensions, tested on twenty of your own questions. Test before you commit: changing model later means re-embedding the whole corpus.

What does this cost to run?

Embedding a corpus is cheap and mostly one-off, a few cents per thousand chunks. The recurring bill is storage, memory for the index, and the model calls you make on every question.

What about documents in other languages?

Use a multilingual embedding model and test retrieval in each language separately. A general English model degrades quietly rather than failing loudly.

What about images, charts and scans?

Scans need OCR before anything else. A chart with no caption is invisible to search, so give figures a text description at ingestion, or accept that nobody will ever retrieve them.

One more, and it decides your architecture: can the data leave your network? If not, you are choosing a self-hosted model on day one, not after the pilot.

The pipeline you are about to build

1

Load

Five formats in, documents out, metadata attached at source.

2

Verify

Inspect extraction before indexing. Catch the silent failures.

3

Chunk

Strategy chosen per document type, not one rule for everything.

4

Embed

One model, applied consistently across the whole corpus.

5

Index

Vectors plus metadata, queryable, in a single corpus.

Step 2 is the one that does not appear in tutorials, and the one that will save you on Day 12.

Lab 1

Ingest five file formats into one indexed corpus

75 minutes · work in pairs · training repo, branch lab-01

What to do

- Load all five provided formats
- Inspect what each loader actually returned
- Choose and justify a chunking strategy per type
- Attach source, page and date metadata
- Embed and index into one corpus
- Run five test queries and record results

What to record

- Which format lost the most information
- The worst chunk you produced, verbatim
- One query that should have worked and did not
- Your chunk size and overlap, and why
- Time spent on parsing versus everything else

Lab debrief

1

Worst chunk contest

Each pair pastes their worst chunk into chat. Read the three funniest aloud.

This is the fastest way to make chunking quality feel real.

2

Which format hurt most

Usually scanned PDFs or spreadsheets.

Ask what they would do differently before writing a single line next time.

3

The query that failed

Collect two or three. Do not solve them today.

We fix these in Week 2 with hybrid retrieval and reranking.

Leave the failed queries visible. They become the test set we measure against for the rest of the programme.

Three things to take from today

Retrieval is the bottleneck

When the answer is wrong, look at what was retrieved before you touch the prompt.

Chunking is engineering

Per document type, with metadata attached, verified before indexing. Not a default value.

Similar is not correct

The embedding measures proximity. Everything else you must build.

Before Day 2

Bring your indexed corpus, it is the input to tomorrow. Write down five queries you expect it to answer well and five you expect it to fail. We will find out which you got right.

AI ENGINEER · DATA · WEEK 1, DAY 2

Vector stores, recall and latency

Choosing where your vectors live, and proving the choice was right

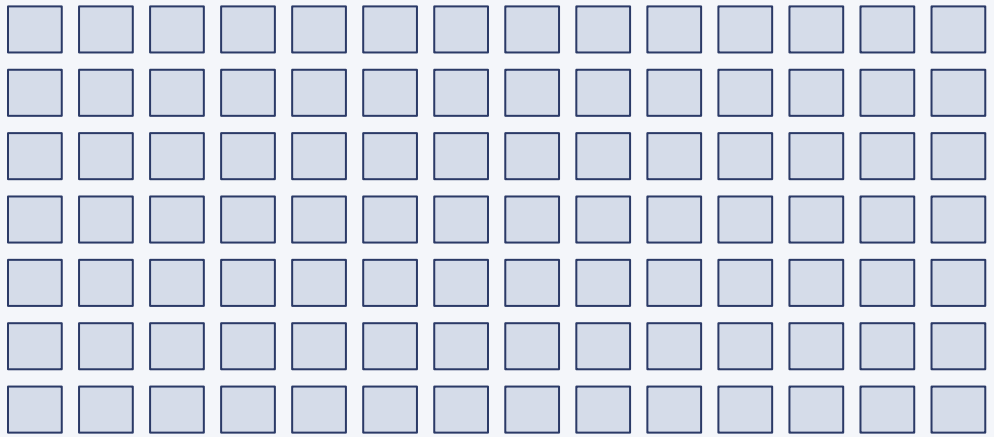
4 hours · instructor-led · benchmarking lab

AN ANALOGY

The warehouse with no aisles

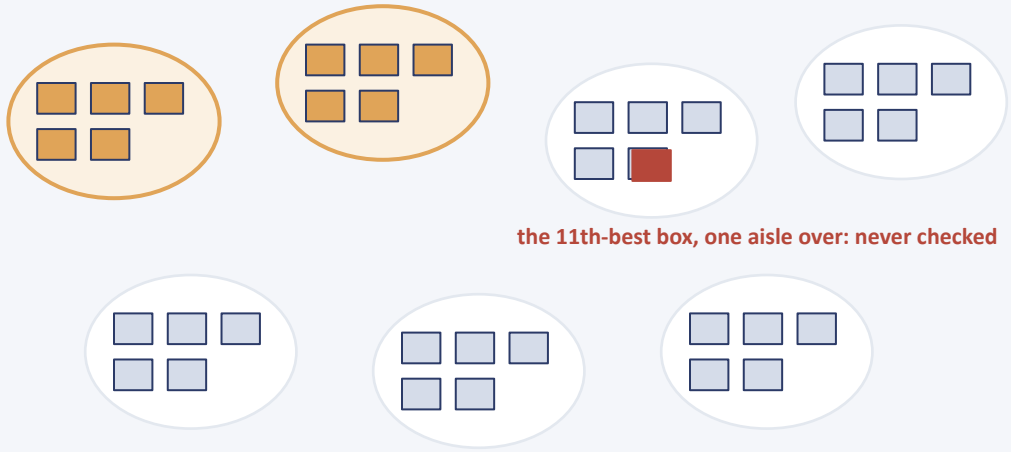
Four million boxes. A customer wants the ten most like the one in her hand.

Open every box



Always right. Takes a week, and longer with every box added.

Signposted aisles



Seconds. Usually the same ten. Usually.

ASK THE ROOM In your business, where would missing the 11th-best result actually matter? Compliance search? Recommendations?

A vector database answers a different question

Ordinary database: exact match

```
WHERE invoice_id = 'INV-2024-88190'
```

INV-2024-88180

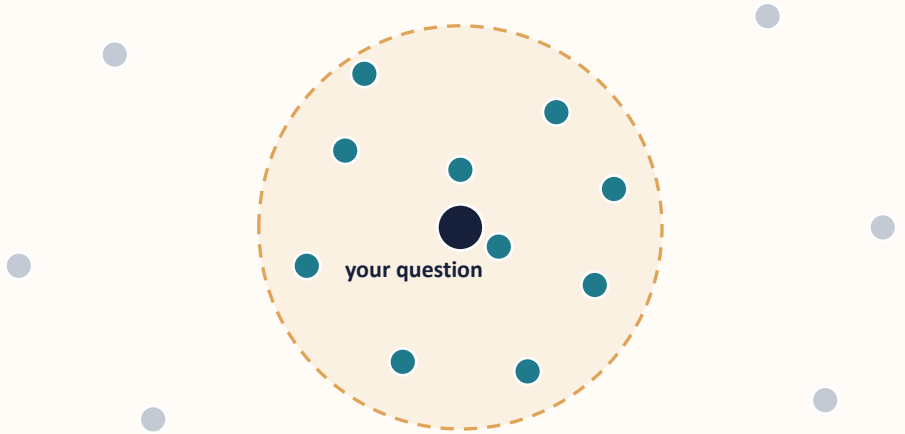
INV-2024-88181

INV-2024-88190 one row, or none

INV-2024-88183

INV-2024-88184

Vector database: closest match



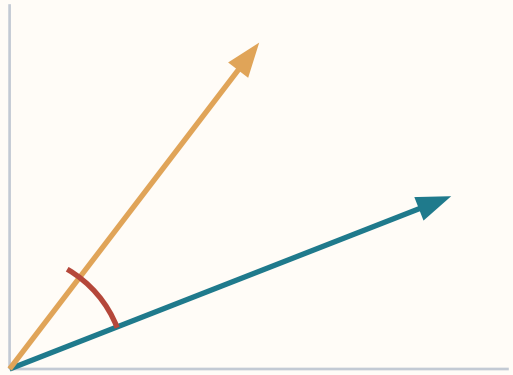
the 10 nearest, ranked

At millions of vectors it cannot compare against everything, so it takes shortcuts. **Shortcuts make it fast. Shortcuts make it miss. Today measures that trade.**

ASK THE ROOM Which of your questions is an exact-match question wearing a semantic-search costume?

How "near" is measured

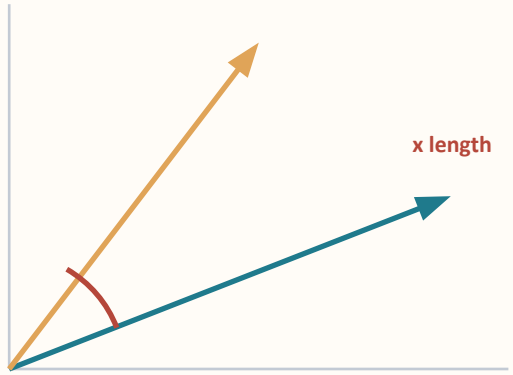
Cosine



The angle between the two arrows. Ignores length.

Most text search

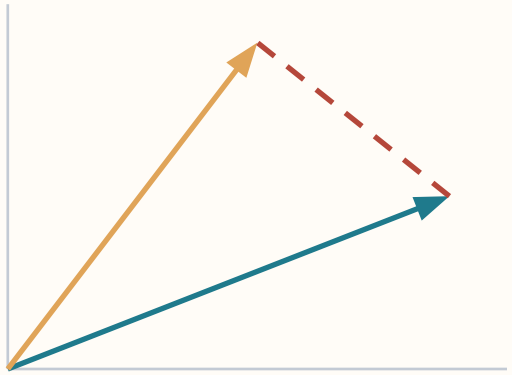
Dot product



Angle and length together. Longer arrows score higher.

Some models are trained for it

Euclidean (L2)



The straight-line gap between the two tips.

Common in image and older indexes

The rule: most text models return vectors scaled to length 1, and then all three give the same ranking. If yours are not, use the measure the model was trained for, and set the SAME one in the index.

ASK THE ROOM Your index is set to Euclidean, your model expects dot product, vectors are not normalised. What breaks, and would anything error?

WHAT A VECTOR STORE ACTUALLY DOES

Four jobs, and only one of them decides enterprise fit

1 STORE

Hold the numbers, the labels, and a pointer back to the source piece.

2 INDEX

Organise them so a search does not have to compare against every one.

3 SEARCH

Return the nearest k to a question, fast, usually approximately.

4 FILTER

Restrict by label (date, department, entitlement) before or during the search.

Job 4 is the one every comparison skips.
Two stores with identical recall behave completely differently once a query carries an entitlement filter that removes 90 percent of the corpus. That case is in Lab 2, and it is usually the surprise of the day.

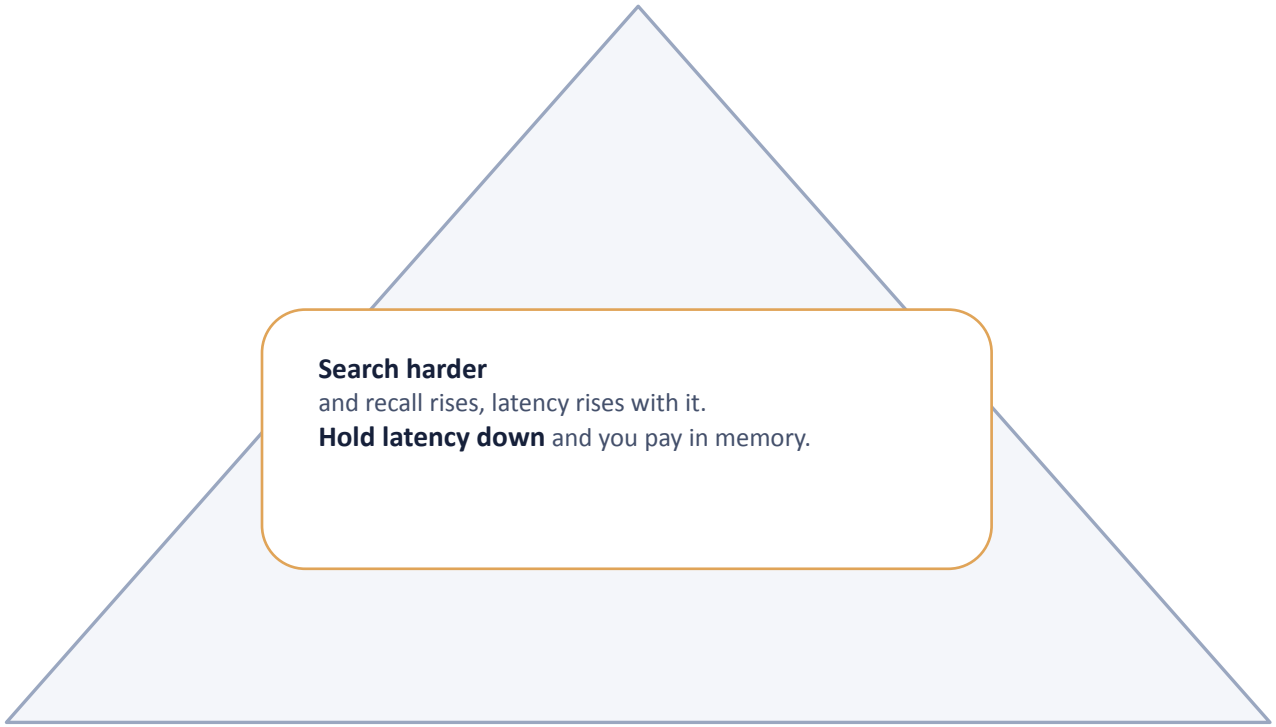
Benchmark the filter, not only the search. Your users never ask an unfiltered question.

THE TRADE YOU ARE ALWAYS IN

Recall, latency and cost: improve any two

RECALL

did you get the right pieces back?



LATENCY

p50 and p95, not the average

COST

memory, storage, build time

Anyone claiming all three at once has changed the question. Ask which one they quietly dropped.

Four ways to find the nearest ticket

THE RUNNING EXAMPLE A FreshCart customer types “The milk leaked all over my order, I want a refund.” The assistant looks for the most similar of 200,000 past tickets.

Exact (flat)

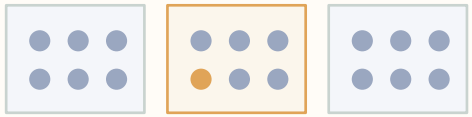


Check every ticket. It reads all 200,000 and keeps the ten closest. Always right; twice the tickets, twice the time.

No dial: it is the answer key for every benchmark.

Speed	slow
Memory	no extra
Accuracy	perfect
Best for	small sets and the answer key

Groups (IVF)

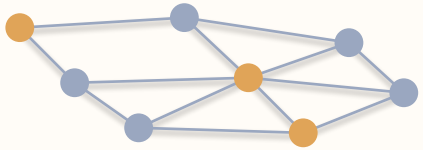


Tickets are sorted into groups ahead of time, by k-means. The question checks the group centres, then searches inside the closest one or two. Fast; it misses a ticket that sits just over the border of a group it skipped.

Dial: how many groups to search.

Speed	fast
Memory	a little extra
Accuracy	good
Best for	large sets on a memory budget

Graph (HNSW)

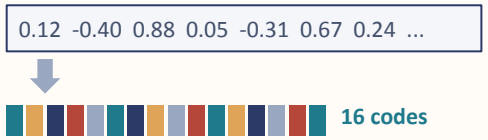


Each ticket is linked to its nearest lookalikes, in layers, like highways, main roads and lanes. The search hops from “app would not let me reorder” to “packaging was torn on arrival” to “spillage across the whole order”. Fast and accurate; the links cost memory.

Dial: search effort (ef), meaning how many candidates it keeps while hopping.

Speed	fastest
Memory	the most
Accuracy	very high
Best for	most production search

Compressed (PQ)

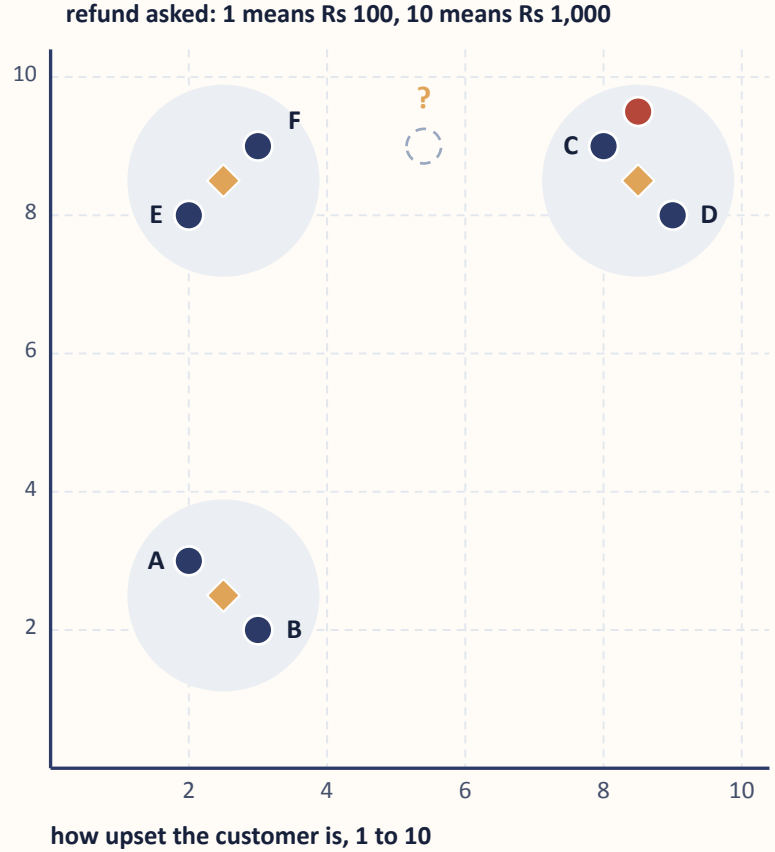


Each ticket’s 1,536 numbers become 16 small codes, and the search compares codes. Hundreds of times less memory; it loses fine detail, so the top few are re-checked with the full numbers. Usually added on top of groups or a graph, not used alone.

Dial: how many to re-check.

Speed	fast
Memory	the least
Accuracy	lower
Best for	very large sets where memory is the bill

How the groups are made: k-means in four steps



THE SIX TICKETS

- **A** wrong size delivered (2, 3)
- **B** packaging was torn on arrival (3, 2)
- **C** spillage across the whole order (8, 9)
- **D** refund has not reached my account (9, 8)
- **E** charged twice for one order (2, 8)
- **F** item missing from the bag (3, 9)

◆ a centre after step 3

● the milk message (8.5, 9.5)

Real tickets have 1,536 scales, not 2, and thousands of groups; the four steps are the same.

1 Guess

Drop three centres anywhere: (1, 2), (8, 8), (2, 7).

2 Assign

Each ticket joins its nearest centre: A and B; C and D; E and F.

3 Average

Move each centre to the middle of its tickets: (2.5, 2.5), (8.5, 8.5), (2.5, 8.5).

4 Repeat

Repeat steps 2 and 3 until the centres stop moving. Here they already have.

The milk message lands at (8.5, 9.5). Its nearest centre is (8.5, 8.5), so it searches only C and D, and finds “spillage across the whole order”.

ASK THE ROOM A new ticket lands at (5.4, 9). Search one group: which of its two nearest tickets does it miss? Search two groups: what changes?

Compression (PQ): keep a palette, not the paint



THE ANALOGY A paint shop can store every customer's colour to ten decimal places, or keep a box of 256 standard shades and store only the shade number. **A little accuracy lost, nearly all the space saved.**

1 Cut

Each ticket's 1,536 numbers are cut into 16 blocks of 96.



2 Palette

For each block position, k-means finds 256 typical blocks.

3 Code

Each ticket keeps 16 palette numbers, one per block: 16 bytes instead of 6,144.



4 Search

The question stays exact. It is compared with each block's 256 palette entries once, and then each ticket's distance is 16 look-ups added up.

200,000 TICKETS

1.2 GB
as full numbers

3.2 MB
as codes, plus 1.6 MB of palettes

The catch, and the fix

Tickets that differ only in detail get the same codes, so "milk leaked over the order" and "juice spilled in the bag" can look identical.

The fix is re-checking: take the top 100 by codes, then re-rank those 100 with the full numbers.

MEASURED ON THE LAB'S TEST DATA

Qdrant, 100,000 vectors of 128 numbers, 30 Sep

	recall@10	re-checked
4x smaller (int8)	**0.91**	**0.99**
16x smaller (PQ)	**0.38**	**0.56**

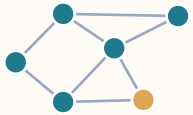
Stacking: IVF-PQ (groups plus codes) is the classic very-large, low-cost setup; Qdrant, Milvus and Azure AI Search add compression to a graph.

ASK THE ROOM Legal searches contracts; marketing searches product reviews. Which one can live with compression?

LAB 2 LINE-UP

The three databases in Lab 2, and where they lead

Chroma



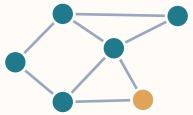
Most named by developers

1.00

recall@10 at ef 40

Inside the notebook. Graph index; set ef when the collection is made.

Qdrant



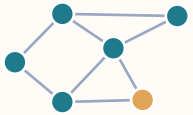
Biggest one-year rise on DB-Engines

1.00

recall@10 at ef 40

Inside the notebook, beta edition. Graph index plus compression.

Milvus



Most GitHub stars

1.00

recall@10 at ef 40

Milvus Lite inside the notebook. About 79 ms a question one at a time, 1.7 ms each in one call.

IN PRODUCTION, THE SAME TRADE WEARS A PRODUCT NAME

pgvector	Azure AI Search	Databricks AI Search	Fabric / Synapse
index: IVFFlat or HNSW	index: HNSW or exhaustive	index: chosen for you	index: chosen for you
accuracy dial: yes: probes, ef_search	**accuracy dial: partly: set in the index**	**accuracy dial: no**	**accuracy dial: no**

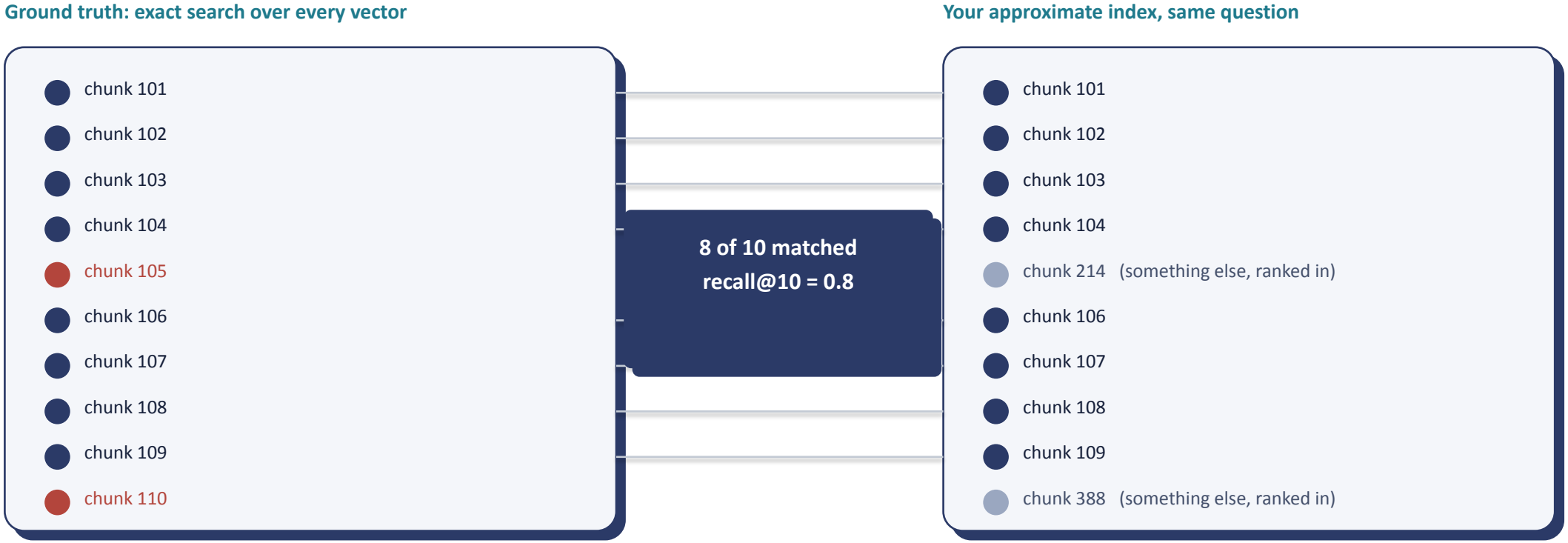
ASK THE ROOM Which of these can you tune yourself, and for the ones you cannot, what must you still measure?

Which store offers which technique

Store	Exact	Groups (IVF)	Graph (HNSW)	Compression	Your dial	In this course
Chroma	no separate mode	no	yes	no	ef_search, set when the collection is made	Lab 2
Qdrant	yes, on request	no	yes	scalar, product, binary	hnsw_ef, per question	Lab 2
Milvus	yes (FLAT)	yes	yes	scalar in Lite; product (IVF_PQ) on the full server	ef or nprobe, per question	Lab 2 (Lite)
pgvector	yes, with no index	yes (IVFFlat)	yes	half-size and bit vectors, no PQ	probes, ef_search	syllabus; Azure track
Azure AI Search	yes (exhaustive)	no	yes	scalar, binary	partly, set in the index	syllabus; Azure track
Databricks AI Search	no	not your choice	yes, on standard endpoints	not your choice	none	Lab 2, trainer-run
Pinecone	no	not published	not published	automatic	none	slide only: needs an account per student

Why these three, 30 Sep 2026: Milvus has the most GitHub stars (46,282), Chroma is the most named in Stack Overflow's 2025 survey (19.7%), Qdrant had the biggest one-year rise on DB-Engines.

Eight of the true ten came back



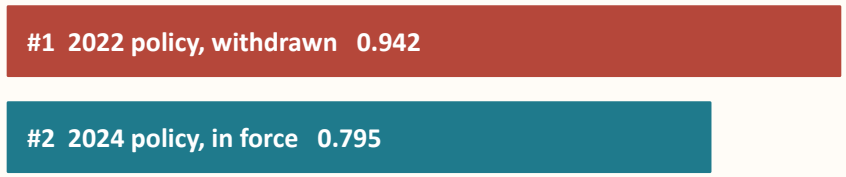
What it does not say: whether those ten chunks were any use to a human. Recall measures the index against exact search, which needs no labelling. Usefulness needs judgement, and that is reranking and evaluation, in Week 2.

Did it find the nearest, or did it find the answer?

Recall@10

Did the index return what exact search would have?

Lab 1 question: how many refund claims in 30 days?

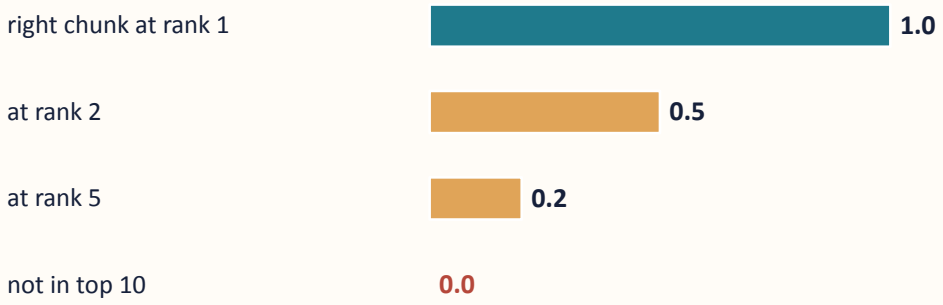


Recall@10 = 1.0: the index matched exact search perfectly.

Needs: nothing labelled. Exact search is the key.
Today, Lab 2

Hit rate and MRR

Did the RIGHT piece come back, and how high?



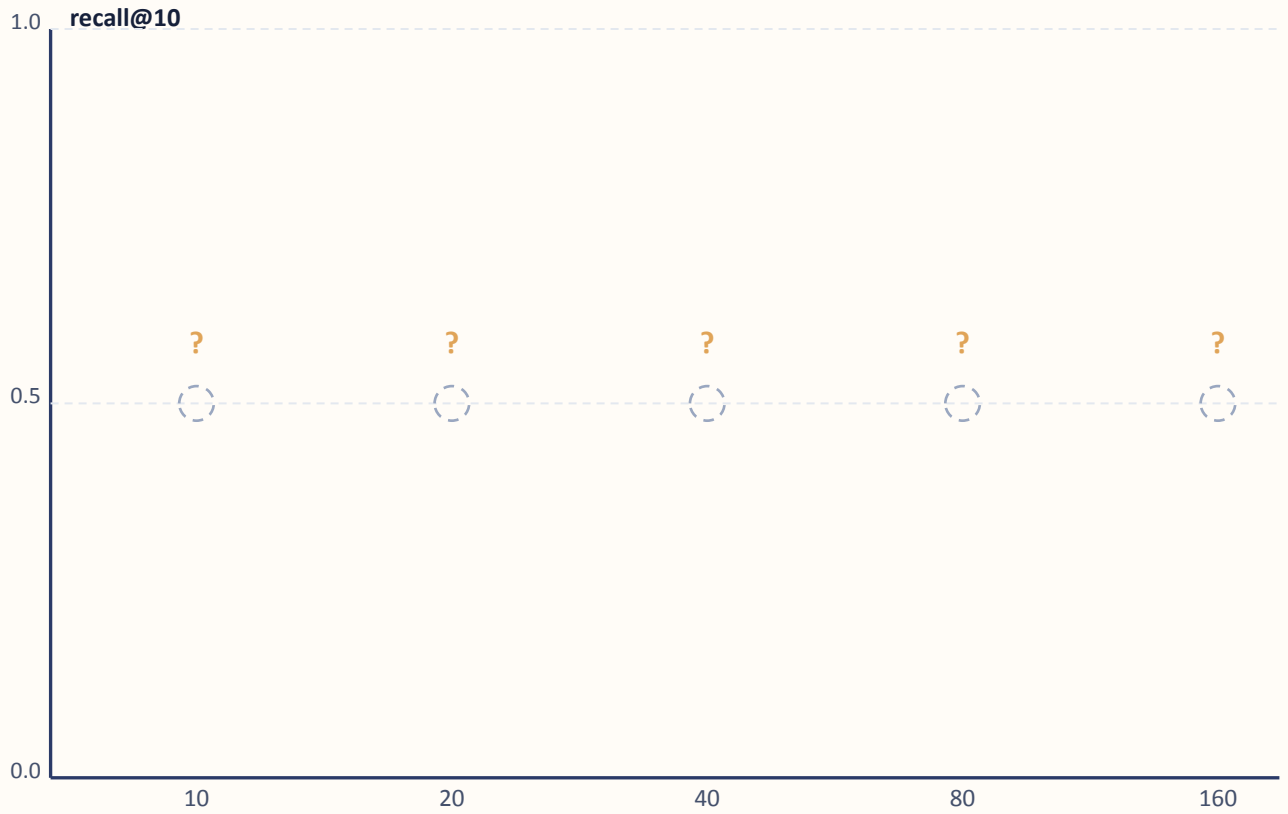
MRR = the average of 1 / rank

Needs: questions with their right chunks, written by people.
Week 2 onwards

ASK THE ROOM Who in your team could write the right chunk for 50 real questions, and how many hours would it take?

DRAW YOUR CURVE FIRST

Predict before you measure



1 Recall

at ef 10? at 160? Where does it stop climbing?

2 p95 latency

Up, down, or flat as the dial turns?

3 Memory

Does it change at all? Why or why not?

4 The elbow

Past which ef are you buying time and no recall?

ASK THE ROOM Type your four predictions in chat now. We reveal the real curve in the debrief, next to your guesses.

QUICK CHECK

Before the break, three questions

1. Your approximate index returns 8 of the true top 10 for the average query. What is that number called, and how did you get the "true" top 10 without anyone labelling data by hand?

2. Suppose your index scores a perfect recall@10 of 1.0 on a question. Can the answer still be wrong? How?

3. A vendor benchmark reports 12 ms average latency. Why is that number, on its own, close to useless?

Take answers in chat. Do not move on until at least two people have committed to an answer.

ANSWERS

Before the break, three questions

1

The 8 out of 10

recall@10 of 0.8. The ground truth comes from exact flat search over the same vectors: correct by definition, no human judgement needed. It measures whether the index found the nearest vectors, not whether those vectors were useful.

2

Perfect recall, wrong answer

Yes. Recall only checks your index against exact search. In Lab 1, exact search itself ranked the withdrawn 2022 policy first, so a perfect score still returned the wrong document. Checking the answer is right needs labelled questions: hit rate and MRR, from Week 2.

3

The 12 ms average

Averages hide the tail your users actually feel, so report p95. It also says nothing about recall: a system can be fast and wrong. And without a stated query set and warm-up, it is not comparable to anything.

Break

10 minutes. After the break: how to run a benchmark whose results you can defend.

5

Benchmarking

Producing a number your architect will accept

SIX STEPS

A benchmark you can defend

1

Fix the corpus

Same chunks, same embedding model, same vectors for every store. Change one thing at a time.

2

Build a query set

At least 50 real queries. Include the ones that failed in Lab 1.

3

Get ground truth

Flat exact search over the same vectors. Store the true top k per query.

4

Warm up, then measure

Discard the first runs. Cold caches make any store look bad.

5

Sweep one parameter

Vary ef alone. Record recall, p50, p95 at each point.

6

Report the curve

Not a single number. The recall-versus-latency curve is the finding.

How benchmarks lie

The mistake	What you actually measured
Different embeddings per store	You measured the embedding model, not the store
Query set too small or too easy	Ten happy-path queries prove nothing about production
No warm-up	You measured cold-cache behaviour and called it latency
Reporting only the average	The p95 is where your users live
Tuning one store and not the others	A default-configured competitor is not a comparison

If your benchmark makes one option look dramatically better, assume you made one of these mistakes and check before you present it.

- Vendor benchmarks are not fabricated. They are tuned by people who know their own product best and yours not at all
- The only benchmark worth acting on runs on your corpus, your queries and your infrastructure

Ask for ten, get three, and nothing reports a problem

POST-FILTER — search first, then throw away what the user may not see

search returns 10 → permissions drop 7 →

user gets

A half-empty answer. The count is wrong, the log says success, and nobody notices for weeks.

PRE-FILTER — narrow the candidates first, then search inside them

allowed set → search inside it →

user gets

Ten results, all of them allowed.

Test it in Lab 2: run a query with a filter that excludes 90 percent of the corpus, and compare the result count and the latency across the three stores. This separates them more than any recall number.

What you will not see until month three

Reindexing

New embedding model means rebuilding everything. How long, and can you serve traffic while it runs?

Updates and deletes

Documents change. Some indexes handle incremental updates gracefully. Others degrade until rebuilt.

Backup and recovery

Can you restore the index, or must you re-embed from source? One is minutes, the other is days.

Ask every vendor and every open-source option these three questions before you ask about recall.

- The GDPR-style question worth rehearsing now: a customer asks for deletion. Can you remove their data from the index, provably, today?

QUESTIONS THAT ALWAYS COME UP

Day 2: before you pick anything

Which vector store should we buy?

The one your platform team can run, out of the ones that pass your filter test at your recall target. Operations decide this more often than milliseconds do.

Do we need a vector database at all?

Below roughly 100,000 vectors, exact search or Postgres with pgvector is genuinely fine. Ask what the number will be in a year, then decide.

When does HNSW start to matter?

Around a million vectors, or sooner if your p95 latency target is tight. Measure at your size, not at the size in the vendor's chart.

How often will we rebuild the index?

Whenever the embedding model changes, which is a full rebuild, and continuously as documents change, which is not. Ask every vendor how each one behaves.

Can we delete one person's data from the index?

You must be able to prove it, today, on request. Test deletion before you buy; Week 6 measures whether it really left every store.

What does hosting actually cost?

Memory is the driver: a million vectors of 1,536 numbers is about 6 GB before overhead, and high-recall graph indexes want it in RAM. Add replicas and you have your bill.

Bring the answer to the first one back as a memo, not a number. The curve, the filter behaviour and the operational risk are the argument; the recall figure alone is not.

QUICK CHECK

Three questions before the lab

1. You benchmark three stores and one comes out dramatically ahead. What should you check before you present that result to anyone?

2. A query carrying an entitlement filter returns 3 results when the user asked for 10. What is happening, and which design choice causes it?

3. Your architect asks which store you should buy. Why is a single recall number the wrong thing to hand over?

Take answers in chat. Do not move on until at least two people have committed to an answer.

Three questions before the lab

1

The suspiciously good result

Check the comparison was fair: identical vectors and embedding model everywhere, warm-up runs discarded, every store tuned rather than one left on defaults, and a query set that reflects real use.

2

Ten asked for, three returned

Post-filtering. The search returned 10, then 7 were dropped for permissions: a half-empty answer that nothing reports. Pre-filtering narrows the candidates first and returns the right count.

3

Why one number is not an answer

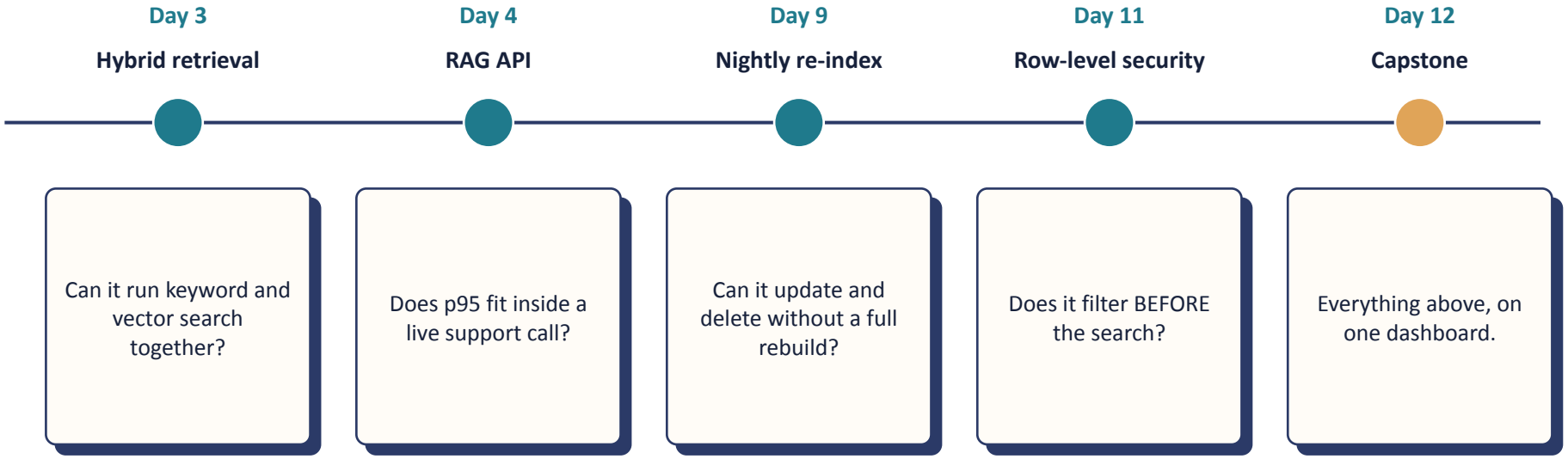
The decision is a curve plus constraints: the recall-versus-latency curve at your target, filter behaviour, the risks around reindexing, deletion and backup, and what would change your mind.

Answer 3 is the shape of the Lab 2 memo you are about to write.

The store you pick today carries the programme

LAB 2

Your store and its operating point



Your syllabus says it plainly: benchmark three stores, pick one for the programme. A weak memo today is a rebuild in Week 5.

ASK THE ROOM Which of these five questions could today's benchmark NOT answer? Put it in the memo as a risk.

LAB 2

Benchmark three databases, then defend one in a memo

80 minutes · in pairs · notebooks/lab02_vector_stores.ipynb

WHAT YOU DO

1. Load the FreshCart pieces, then scale to 100,000 vectors
2. Build the answer key with exact search
3. Measure Chroma, Qdrant and Milvus: recall@10, p50, p95, load time, one question at a time and 200 in one call
4. Sweep ef over 10, 20, 40, 80, 160, predicting the shape first
5. Compression and the filter test
6. Measure the trainer's Databricks AI Search index, then write the memo

DONE MEANS

- All three databases measured on every number in step 3
- The ef curve, with the elbow marked
- The filter counts, and the difference explained
- The "Your write-up" cell finished: which database, at which operating point, and what would change your mind

Predict before you measure. Compare your curve with the predictions you typed in chat before the break. The gap between your guess and the curve is the part you will remember.

Lab debrief and the re-vote

1

The curves

Two or three pairs share their recall-versus-latency curve.

Where was the elbow? Did anyone find ef stopped helping?

2

The filter test

Which store degraded under the selective filter?

Usually the most surprising result of the day.

3

The re-vote

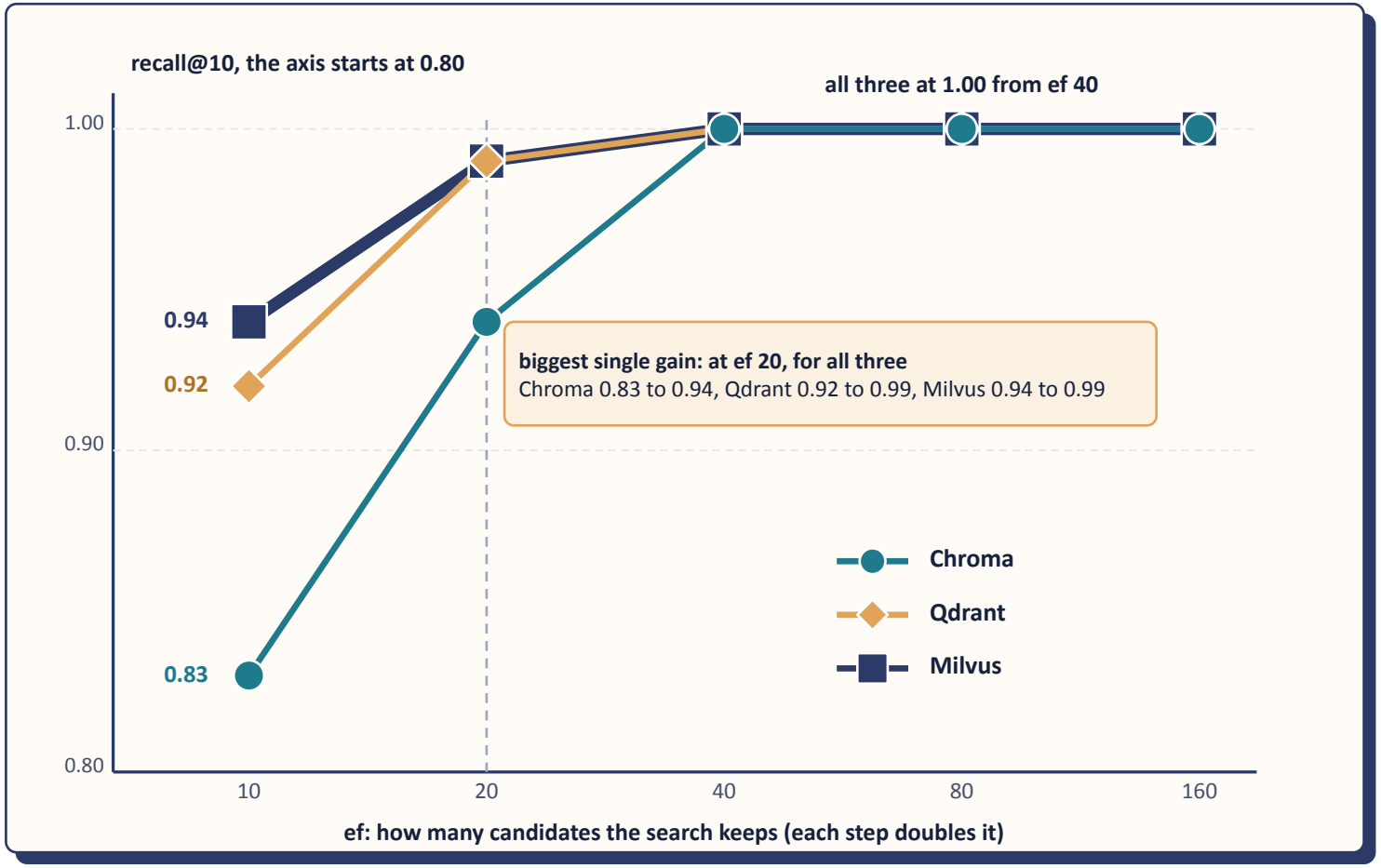
Back to yesterday's question: what breaks first?

Compare against the Day 1 tally. The shift is the learning.

Disagreement between pairs is useful. Ask the two who chose differently to defend their operating points against each other.

DEBRIEF REVEAL

The real curve, next to your guess



MEMORY

unchanged

ef is used when a question is asked, not when the index is built.

LATENCY

79 ms on Milvus Lite

per question at ef 40, asked one at a time, against about 0.28 ms on Qdrant and 2.3 ms on Chroma. Sent 200 in one call: 1.7 ms each.

FILTER TEST

0.8 of 10

returned on average when filtering after the search. Each database's own filter: 10 of 10.

ASK THE ROOM Whose prediction was closest? Now: at what ef would YOU ship, and what target made you pick it?

Week 1 in three sentences

You built the foundation

A corpus ingested from five hostile formats, chunked deliberately, with metadata attached.

You can prove it works

Ground truth, recall, latency percentiles, and a decision memo you could defend to an architect.

You found the gaps

The queries that still fail. Acronyms, identifiers, stale documents. All of them real.

Week 2: hybrid retrieval, cross-encoder reranking, metadata filtering and grounding with citations. We take the queries that failed this week and fix them, then measure the lift.